# CSC3002
# C/C++ Programming

## Introduction to C++

**Prof. Dr. Ruizhi Chen，SDS/SAI**
**Office: TxC 717**

# Grading

- Midterm: 25%(Easy)
- Final Exam: 45% (Difficult)
- Assignments: 30%  (Normal)
- Both mid-term and final are closed-book exams.

# Assignments

1. Three simple coding assignments (20% each， No AI allowed)

2. One coding project using AI (40%)

# Teaching Team

Instructor: Prof. Ruizhi Chen (chenruizhi@cuhk.edu.cn TxC 717)

TA Teams:

Zhaoyang Zhang (Lead TA) 225040464@link.cuhk.edu.cn

Zhenrui Zheng 225040512@link.cuhk.edu.cn

Mocheng Li mochengli1@link.cuhk.edu.cn

Yuxin Peng yuxinpeng@link.cuhk.edu.cn

Geoffrey Mikhael 124040051@link.cuhk.edu.cn

# Relations with other courses

There is no needed for students of SDS/CS and SAI to take this course, because you will have to take CSC3200 as a compulsory course in your study plan. CSC3200 is an advanced version of this course designed specially for computer science and AI.

If you have taken the course of CSC3100, the data structure part is similar. Please consider to take this course or not.

# Attendance of the class

We will record the attendance of the class. However, class attendance WILL NOT affect your final score. This will be the information for the instructor only (the see the attendance rate for difference parts of the lecture contents)

# Issues about AI coding -- Standford

- **Foundational courses (CS106B – C++/Data Structures):** Strictly restrict AI use. Emphasize in-person “code interviews” and heavily weigh final exams. The philosophy: struggling with fundamentals is essential for learning.

**CSC3002 is copied from Standford CS106B**

- **Advanced courses (CS146S – Modern Software Development):** **Mandate** AI usage (Copilot, ChatGPT, etc.) – no hand-written code allowed. Students submit their chat logs with AI agents as part of assignments.

# Issues about AI coding – UC Berkeley

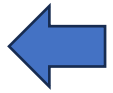
- **CS 61A (Python introduction):** Since Fall 2026, allowed AI-coding agents in the final project for building complex applications.
- **Early findings:** Students over-relying on AI showed a **significant rise in failure rates** (e.g., 10.6% in CS 61A), and those who used AI during coursework performed worse on handwritten final exams.
- **Differentiated policies:** Stricter AI bans in lower-division courses, but more permissive in upper-division or project-based classes (e.g., “Generative Systems Design”).

# Issues about AI coding – MIT

- **No uniform policy:** Each instructor decides; default rule is **AI is prohibited** unless explicitly allowed in the syllabus.
- **Varied practices:** Some introductory courses allow AI for explanation but not for generating final code, with mandatory “code defense” sessions. Others ban AI entirely.
- **Assessment overhaul:** Recommendations to reduce traditional homework weight and adopt **oral exams, project presentations, and live debugging** to evaluate genuine understanding.

# Why we need to learn C++

*Implement two swap functions, one is pointer-based, while the other is reference-based, with a demo and store them in the file of swap.cpp*



**We need knowledge to tell AI what to code, in which way to create the most efficient codes**

## Pointer-based

```
void swap(int* first, int* second)
{
    int temporary = *first;
    *first = *second;
    *second = temporary;
}
```

## Reference-based

```
void swap(int& first, int& second)
{
    int temporary = first;
    first = second;
    second = temporary;
}
```

# Why we need to learn C++

*Implement a swap function using template*

*Implement a swap function using smart pointer*

## Using template

```
template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
```

## Using smart pointer

```
void swap(std::unique_ptr<int>& a,\
std::unique_ptr<int>& b)
{
    a.swap(b);
}
```

# Reference Text Book




Eric S. Roberts
Stanford University
Spring Quarter 2012

# Course Contents

**Part I: C++ Programming (5 Weeks)**

Data types, function, memory management, class, template.

**Part II: Introduction to Data Structures (5 weeks)**

vector, linked list, stack, queue, priority queue, set, map/hashing map, tree, BST, heap, graph.

**Part III: Algorithms (3 weeks)**

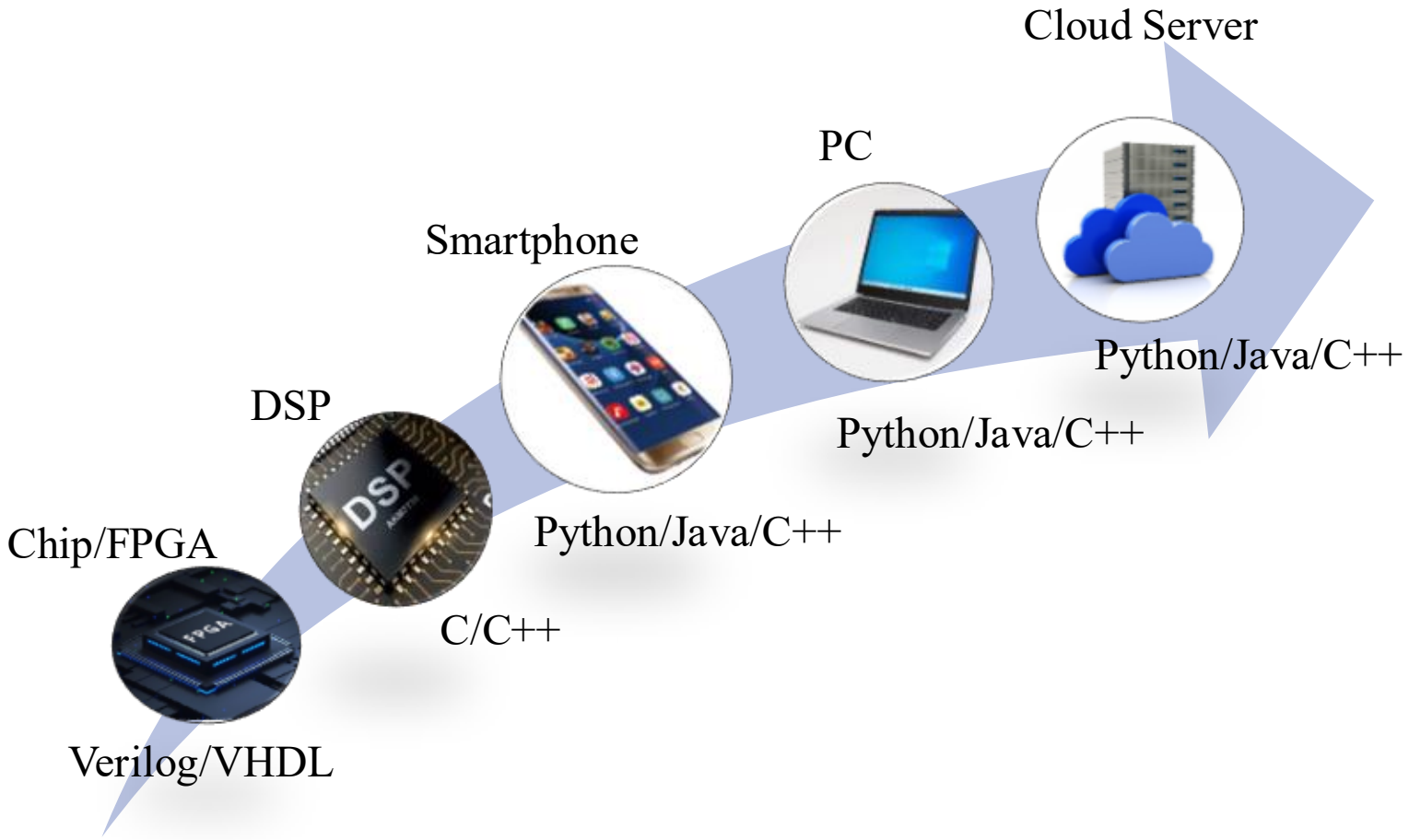
- Algorithm Analysis
- Sorting
- Recursion
- Backtracking

**Part IV: Course review (1 week)**

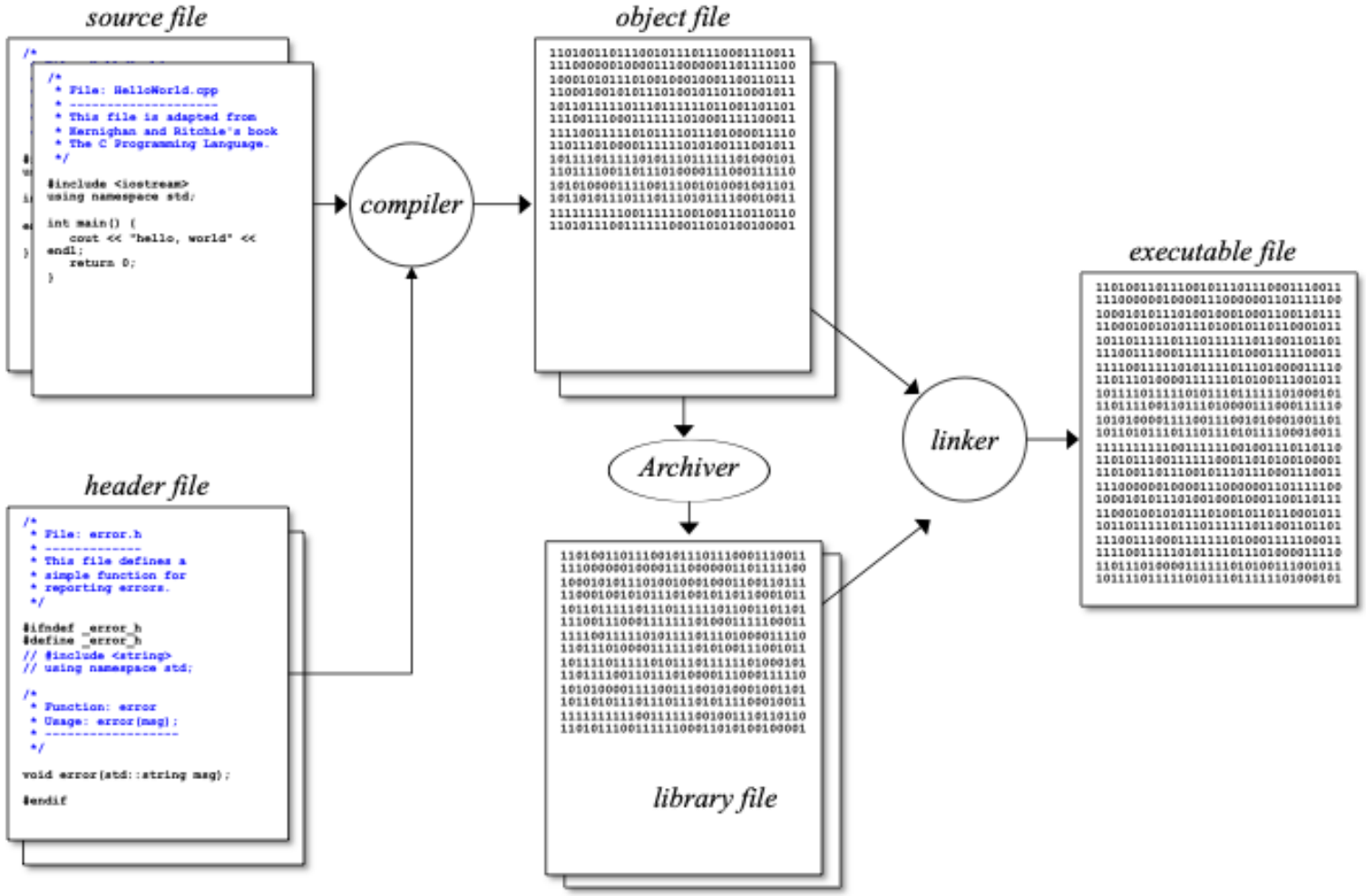
# An Overview of C++

1.1 The compilation process
1.2 Your first C++ program
1.3 The structure of a C++ program
1.4 Variables
1.5 Data types
1.6 Expressions
1.7 Statements

# Hardware Platform vs Programming Language



# The Compilation Process




#create a object file.

g++ -c Math1.cpp -o Math1.o
g++ -c Math2.cpp -o Math2.o

#generate the lib, r-replace,
# c-create, s-symbol table)

ar rcs MyMath.lib Math1.o Math2.o

#create App using lib

g++ main.cpp -L. MyMath.lib -o main

```cpp
#include <iostream>

using namespace std;

int main() {

  cout << ”Hello World" << std::endl;

  return 0;
}
```



```
> Hello World
>
```

# The Structure of a C++ Program

- A comment is text that is ignored by the compiler but which nonetheless conveys information about the source code to other programmers (actually, to yourself too).

- Multi-line comments

```
/* Comments line 1
   Comments line 2
   Comments line 3 */
```

- Single-line comments

```
// Comments line 1
// Comments line 2
// Comments line 3
```

- It is extremely important to write comments.  Code tells you how, comments tell you why!

# The Structure of a C++ Program

```
#include <iostream>
using namespace std;

/* Function prototypes */

int raiseToPower(int n, int k); // Declare function.

/* Main program */

int main() {
   int limit; // Declare before using it
   cout << "This program lists powers of two." << endl;
   cout << "Enter exponent limit: ";
   cin >> limit;
   for (int i = 0; i <= limit; i++) {
      cout << "2 to the " << i << " = "
           << raiseToPower(2, i) << endl;
   }
   return 0;
}
```

```
/*
 * Function: raiseToPower
 * Usage: int p = raiseToPower(n, k);
 * ----------------------------------
 * Returns the integer n raised
 */

int raiseToPower(int n, int k) {
   int result = 1;
   for (int i = 0; i < k; i++) {
      result *= n;
   }
   return result;
}
```

# Namespace vs Library

## Namespace

A namespace is a container for organizing identifiers like functions and variables to prevent name conflicts, ensuring clarity and structure in your code

## Library

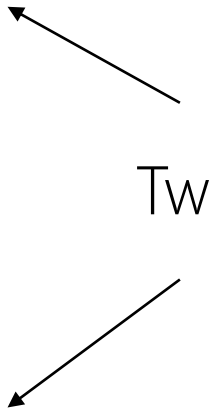
A library is a collection of pre-written code—such as functions or modules—that you can reuse to add specific functionality to your projects

**Namespaces focus on organization, and libraries focus on providing reusable code.**

# Namespace – header files

```
//Math1.h
#pragma once
namespace math1
{
        int add(const int x, const int y);
        int sub(const int x, const int y);
};
```

```
//Math2.h
#pragma once
namespace math2
{
        double add(const double x, const int y);
        double sub(const double x, const int y);
};
```



Two namespaces

# Namespace – source files

//Math1.cpp

```
#include "Math1.h"

int math1::add(const int x, const int y)
{
		return x + y;
}

int math1::sub(const int x, const int y)
{
		return x - y;
}
```

//Math2.cpp

```
#include "Math2.h"

int math2::add(const double x, const int y)
{
		return x + y;
}

int math2::sub(const double x, const int y)
{
		return x - y ;
}
```

# Namespace –Main file

```
#include <iostream>
#include "Math1.h"
#include "Math2.h"

int main()
{
	int x = 100;
	int y = 10;
	double xx = 100.00;
	Double yy = 10.0;
	std::cout << "add = " << math1::add(x, y) << std::endl;
	std::cout << "sub = " << math1::sub(x, y) << std::endl;
	std::cout << ”add = " << math2::add(xx, yy) << std::endl;
	std::cout << ”sub = " <<math2::sub(xx, yy) << std::endl;

	return 0;
}
```

# Makefile

```
#!/bin/bash

#create a object file.

g++ -c Math1.cpp -o Math1.o
g++ -c Math2.cpp -o Math2.o

#generate the lib, r-replace, c-create, s-symbol table)

ar rcs MyMath.lib Math1.o Math2.o

#create App using lib

g++ main.cpp -L. MyMath.lib -o main
```

Two namespaces in one LIB

# Data Types

- A data type is defined by a domain, which is the set of values that belong to that type, and a set of operations, which defines the behavior of that type.
- Although many data types are represented using object classes or other compound structures (more details later), C++ defines a set of primitive types to represent simple data.
- Of all the primitive types available in C++, the programs in this text typically use only the following four:

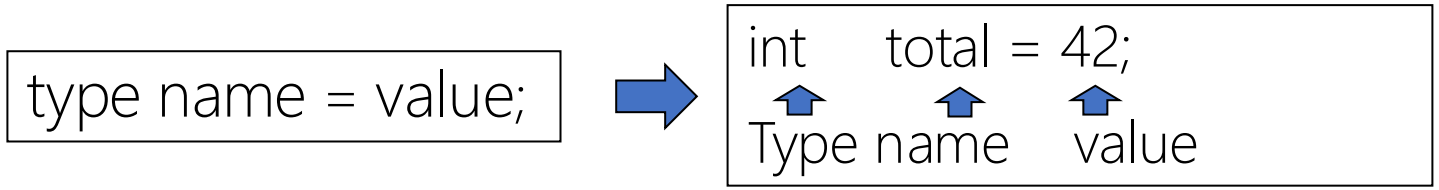
| | |
|---|---|
| int | This type is used to represent integers, which are whole numbers such as 17 or –53. |
| double | This type is used to represent numbers that include a decimal fraction, such as 3.14159265. |
| bool | This type represents a logical value (true or false). |
| char | This type represents a single ASCII character. |

```
struct person {
char name[20];
int      age;
DoB   date-of-birth;
 };
```

```
struct DoB {
int            year;
int           month;
int             day;
 };
```

# Variables

- A variable is a named address for storing a type of value.
- In C++, you must declare a variable before you can use it. The declaration establishes the name and type of the variable and, in most cases, specifies the initial value as well.
- The most common form of a variable declaration is

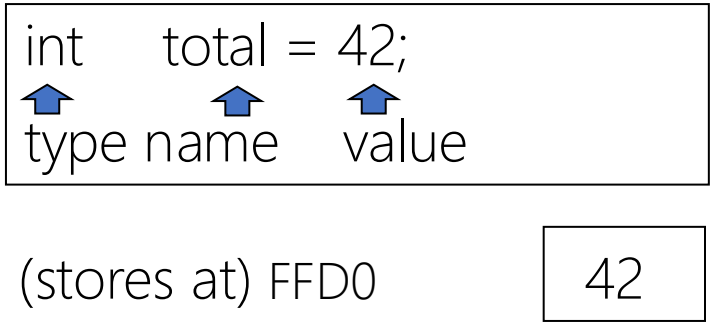



where type is the name of a C++ primitive type or more complicated type such as class, name is an identifier that indicates the name of the variable, and value is an expression specifying the initial value.

- Through the declaration, a memory address is associated with the variable name, which we will discuss later in memory management.

# Variables

- A variable in C++ is most easily envisioned as a box capable of storing a value. For the following statement:




- Each variable has the following attributes:
  - A name, which enables you to differentiate one variable from another.
  - A type, which specifies what type of value the variable can contain.
  - A value, which represents the current contents of the variable.
  - For now, let's not worry about the address first.
- The address and type of a named variable are fixed. The value changes whenever you assign a new value to the variable.

# Variables

- Variable name is case sensitive.
- Naming conventions (also a part of programming style)
  - The name must start with a letter or the underscore character (_).
  - All other characters in the name must be letters, digits, or the underscore. No spaces or other special characters are permitted in names.
  - Use meaningful words (good variable names are like comments), but must not be one of the reserved keywords.

**TABLE 1-1** **Reserved words in C++**

| | | | | |
|---|---|---|---|---|
| asm | do | inline | short | typeid |
| auto | double | int | signed | typename |
| bool | dynamic_cast | long | sizeof | union |
| break | else | mutable | static | unsigned |
| case | enum | namespace | static_cast | using |
| catch | explicit | new | struct | virtual |
| char | extern | operator | switch | void |
| class | false | private | template | volatile |
| const | float | protected | this | wchar_t |
| const_cast | for | public | throw | while |
| continue | friend | register | true | |
| default | goto | reinterpret_cast | try | |
| delete | if | return | typedef | |

# Variable Scope and Extent

- Most declarations appear as statements in the body of a function definition. Variables declared in this way are called local variables and are accessible only inside that function. They are also sometimes called automatic variables because they are automatically created when the function starts execution, and automatically go away when the function is finished executing.

```
if(a==x) {
int age = 0;
age ++;
} //after running this block of code, “age” will disappear
```

- Variables may also be declared as part of a class. These are called instance variables and are covered in later chapters.
- Variables declared outside any function and class definition are global variables. Avoid global variables because they can be manipulated by any function in a program, and it is difficult to keep those functions from interfering with one another.

# Variable Scope and Extent

- In computer programming, a name binding is an association of a name to an entity (e.g., data), such as a variable.
- The scope of a name binding is the region of a computer program where the binding is valid, i.e., where the name can be used to refer to the entity. From the perspective of the referenced entity, it is also known as the visibility of the entity.
- While the scope of a variable describes where in a program's text the variable may be used, the extent (or lifetime) describes when in a program's execution a variable has a (meaningful) value.

# Constants

- Unlike a variable, which is a placeholder for a value that can be updated as the program runs, the value of a constant does not change during the course of a program
- The format of a constant depends on its type:
  - Integral constants consist of a string of digits, optionally preceded by a minus sign, as in 0, 42, -1, or 1000000.
  - Floating-point constants include a decimal point, as in 3.14159265 or 10.0. Floating-point constants can also be expressed in scientific notation by adding the letter E and an exponent after the digits of the number, so that 5.646E-8 represents the number $5.646 \times 10^{-8}$.
  - The two constants of type boolean are true and false.
  - Character and string constants are discussed in detail in later lectures. For the moment, all you need to know is that a character constant is enclosed in single quotation marks, such as 'a', while a string constant consists of a sequence of characters enclosed in double quotation marks, such as "hello, world".

# Named constants

- If you use the same constant many times in a program, it would be better if you could give this constant a name and then refer to it by that name everywhere in the program.
- To create a named constant in C++, precede the regular variable declaration with the keyword const. This tells the compiler that a special variable has been created which has a value that is invariable.

```
const double PI = 3.14159265358979323846;
```

- When creating a named constant, it must be assigned a value. You cannot assign a value to a named constant after its initialization.

```
PI = 10.0; // Error
```

# Named constants

- In C, the preprocessor directive #define was used to create a name for a constant so that the preprocessor can replace every instance of the name with the appropriate constant throughout the code.

```
#define PI 3.14159265358979323846
```

- This still works in C++, but since a constant created this way with a scope from the point of definition to the end of the file, it is possible to have the same definition in another file with a completely different value. This could lead to disastrous consequences.
- Using const is a good habit because we can control its scope.

# Expressions

```
/*
 * File: AddThreeNumbers.cpp
 * -------------------------
 * This program adds three floating-point numbers and prints their sum.
 */

#include <iostream>
using namespace std;

int main() {
   double n1, n2, n3;
   cout << "This program adds three numbers." << endl;
   cout << "1st number: ";
   cin >> n1;
   cout << "2nd number: ";
   cin >> n2;
   cout << "3rd number: ";
   cin >> n3;
   double sum = n1 + n2 + n3;
   cout << "The sum is " << sum << endl;
   return 0;
}
```

# Expressions

- The heart of the AddThreeNumbers program is the line

```
double sum = n1 + n2 + n3;
```

  that performs the actual addition.

- The n1 + n2 + n3 that appears to the right of the equal sign is an example of an expression, which specifies the operations involved in the computation.
- An expression in C++ consists of terms joined together by operators.
- Each term must be one of the following:
  - A constant value (such as 3.14159265 or "hello, world")
  - A constant/variable name (such as PI, n1, n2, or total)
  - A function call that returns a value (such as readInt())
  - An expression enclosed in parentheses ( )

# Operators and Operands

- As in most languages, C++ programs specify computation in the form of arithmetic expressions that closely resemble expressions in mathematics.
- The most common operators in C++ are the ones that specify arithmetic computation:

| | | | |
|---|---|---|---|
| + | Addition | * | Multiplication |
| – | Subtraction | / | Division |
| | | % | Remainder |

- Operators in C++ usually appear between two sub-expressions, which are called its operands. Operators that take two operands are called binary operators.
- The - operator can also appear as a unary operator, as in the expression -x, which denotes the negative of x.

# Division and Type Casts

- In C++, whenever you apply a binary operator to numeric values, the result will be of type int if both operands are of type int, but will be a double if either operand is a double.
- This rule has important consequences in the case of division. For example, the expression

```
14/5
```

  seems as if it should have the value 2.8, but because both operands are of type int, C++ computes an integer result by throwing away the fractional part. The result is therefore 2.
- If you want to obtain the mathematically correct result, you need to convert at least one operand to a double, as in

```
double(14)/double(5)
```

```
double(14)/5
```

```
14/double(5)
```

  The conversion is accomplished by means of a type cast, which you specify by using the type name as a function call.

# Division and Type Casts

- Question: what are the results of the following expressions?

```
double(14/5)
```
2.0

```
double(14)/5
```
2.8

```
14/double(5)
```
2.8

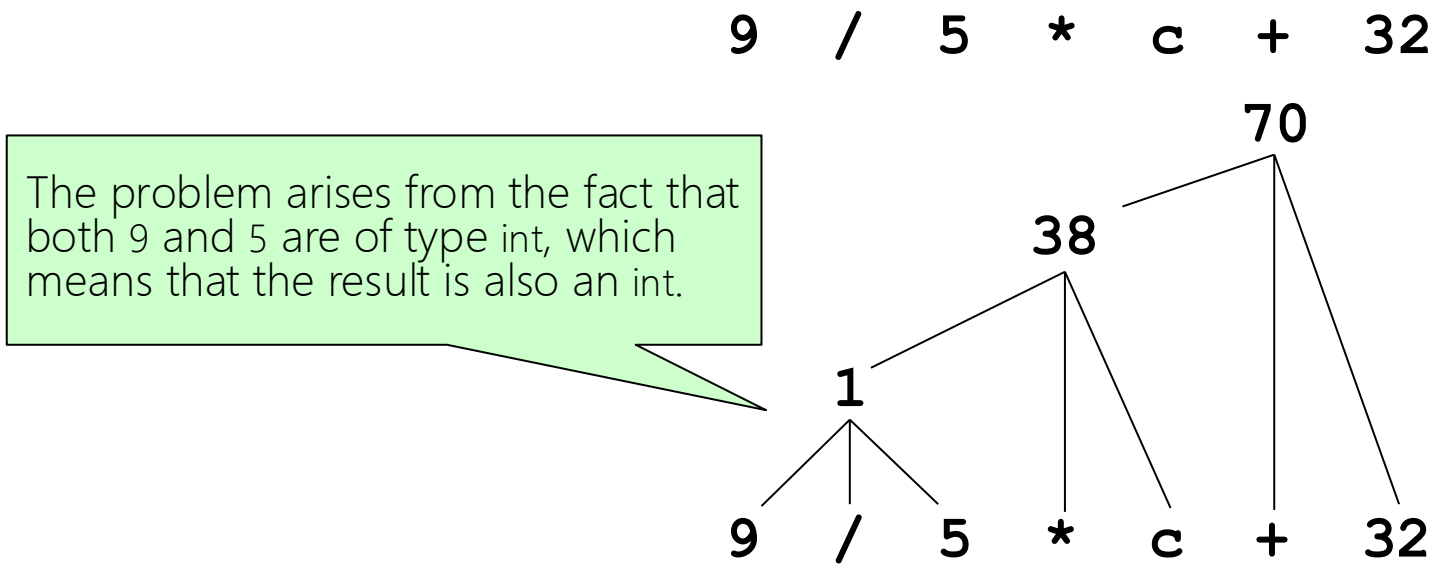
# Exercise: The Pitfalls of Integer Division

Consider the following C++ statements, which are intended to convert 38° Celsius temperature to its Fahrenheit equivalent:

```
double c = 38;
double f = 9 / 5 * c + 32;
```



The computation consists of evaluating the following expression:

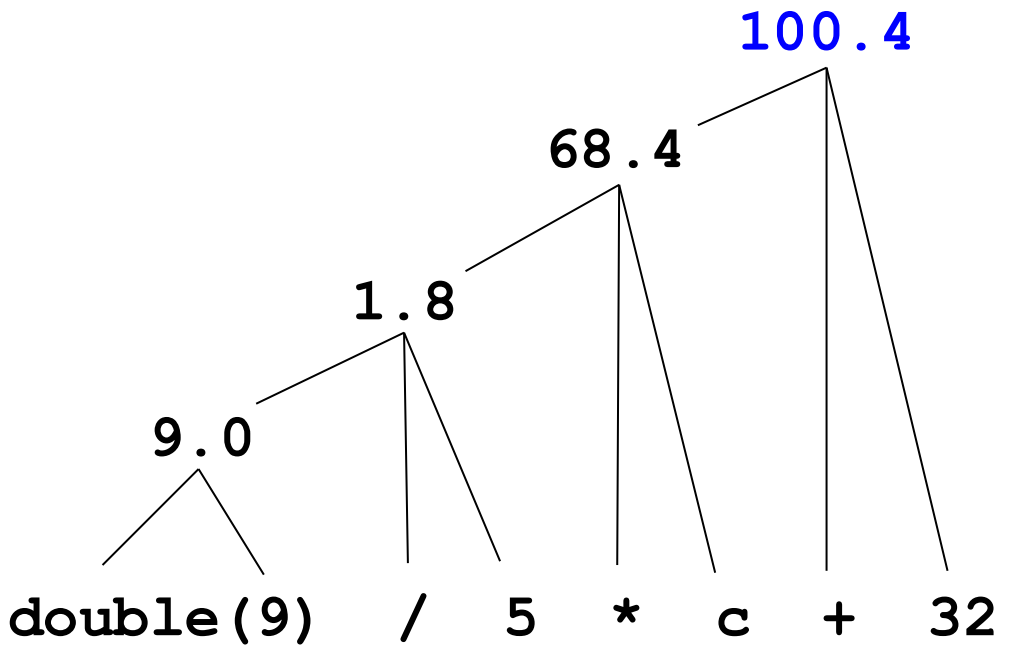


# The Pitfalls of Integer Division

You can fix this problem by converting the fraction to a double, either by inserting decimal points or by using a type cast:

```
double c = 38;
double f = double(9) / 5 * c + 32;
```

The computation now looks like this:



# The Remainder Operator

- The only arithmetic operator that has no direct mathematical counterpart is %, which applies only to integer operands and computes the remainder when the first is divided by the second:

```
14 % 5   returns   4
14 % 7   returns   0
7 % 14   returns   7
```

```
a % b = a – (a/b)*b
```

- The result of the % operator makes intuitive sense only if both operands are positive.

- The remainder operator turns out to be useful in a surprising number of programming applications and is well worth a bit of study.

# Precedence

- If an expression contains more than one operator, C++ uses precedence rules to determine the order of evaluation. The arithmetic operators have the following relative precedence:

**TABLE 1-4** **Operators available in C++**

| Operators organized into precedence groups | Associativity |
|---|---|
| `()` `[]` `->` `.` | *left* |
| *unary operators:* `-` `++` `--` `!` `&` `*` `~` `(`*type*`)` `sizeof` | *right* |
| `*` `/` `%` | *left* |
| `+` `-` | *left* |
| `<<` `>>` | *left* |
| `<` `<=` `>` `>=` | *left* |
| `==` `!=` | *left* |
| `&` | *left* |
| `^` | *left* |
| `\|` | *left* |
| `&&` | *left* |
| `\|\|` | *left* |
| `?:` | *right* |
| `=` *op*`=` | *right* |

# Precedence

- Precedence applies only when two operators compete for the same operand. If two operators have the same precedence, they are applied in the order specified by their associativity, which indicates whether that operator groups to the left or to the right. For example: x = y = z = 0, in the order from right;

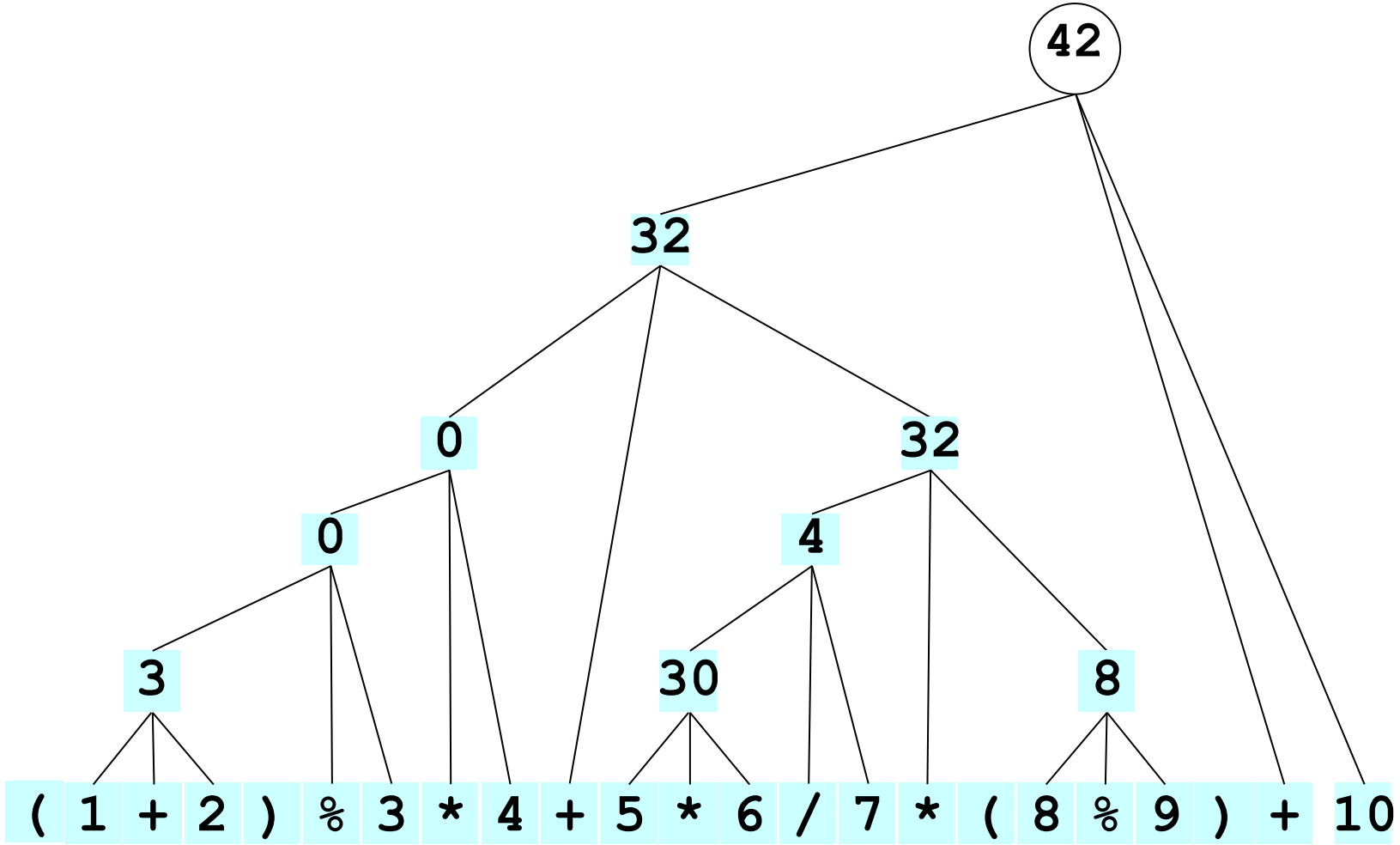
- Most operators in C++ are left-associative, which means that the leftmost operator is evaluated first.

- A few operators – most notably the assignment operator – are right-associative, which means that they group from right to left.

- Parentheses ( ) may be used to change the order of operations. It is always a good habit to use parentheses to clearly identify the precedence and associativity (like writing comments).

# Exercise: Precedence Evaluation

What is the value of the expression at the bottom of the screen?



# Assignment Statements

- You can change the value of a variable in your program by using an assignment statement, which has the general form:

```
variable = expression;
```

- The effect of an assignment statement is to compute the value of the expression on the right side of the equal sign and assign that value to the variable that appears on the left. Thus, the assignment statement

```
total = total + value;
```

  adds together the current values of the variables total and value and then stores that sum back in the variable total.

- When you assign a new value to a variable, the old value of that variable is lost.

# Shorthand Assignments

- Statements such as

```
total = total + value;
```

  are so common that C++ allows the following shorthand form:

```
total += value;
```

- The general form of a shorthand assignment is

```
variable  op=  expression;
```

  where op is any of C++'s binary operators. The effect of this statement is the same as

```
variable = variable op (expression);
```

  For example, the following statement multiplies salary by 2.

```
salary *= 2;
```

# Increment and Decrement Operators

- Another important shorthand form that appears frequently in C++ programs is the increment operator, which is most commonly written immediately after a variable, like this:

```
x++;
```

```
++x;
```

  The effect of this statement is to add one to the value of x, which means that this statement is equivalent to

```
x += 1;
```

  or in an even longer form

```
x = x + 1;
```

- The -- operator (which is called the decrement operator) is similar but subtracts one instead of adding one.
- The ++ and -- operators are more complicated than shown here, but it makes sense to defer the details until later.

# Pre-Incremental and Post-incremental Operators

```
int a = 5;
int b = a++;   // post-increment

// Result:
// a = 6   (it was incremented)
// b = 5   (the old value is returned)
```

```
int a = 5;
int b = ++a;   // pre-increment

// Result:
// a = 6   (it was incremented)
// b = 6   (the old value is returned)
```

# Boolean Expressions

- The operators used with the boolean data type fall into two categories: relational operators and logical operators.
- There are six relational operators that compare values of other types and produce a boolean result:

| | | | |
|---|---|---|---|
| `==` | Equals | `!=` | Not equals |
| `<` | Less than | `<=` | Less than or equal to |
| `>` | Greater than | `>=` | Greater than or equal to |

  For example, the expression `n <= 10` has the value true if `n` is less than or equal to 10 and the value false otherwise.
- There are also three logical operators:

| | | |
|---|---|---|
| `&&` | Logical AND | `p && q` means both `p` and `q` |
| `\|\|` | Logical OR | `p \|\| q` means either `p` or `q` (or both) |
| `!` | Logical NOT | `!p` means the opposite of `p` |

# Notes on the Boolean Operators

- Remember that C++ uses = to denote assignment. To test whether two values are equal, you must use the == operator.
- It is not legal in C++ to use more than one relational operator in a single comparison as is often done in mathematics. To express the idea embodied in the mathematical expression

```
0 <= x <= 9
```



you need to make both comparisons explicit, as in

```
0 <= x && x <= 9
```

- The || operator means either or both, which is not always clear in the English interpretation of or (e.g., exclusive or).
- Be careful when you combine the ! operator with && and || because the interpretation often differs from informal English.

# Short-Circuit Evaluation

- C++ evaluates the && and || operators using a strategy called short-circuit mode in which it evaluates the right operand only if it needs to do so.
- For example, if n is 0, the right-hand operand of the && operator in

```
n != 0 && x % n == 0
```

- is not evaluated at all because n!=0 is false. Because the expression

```
false && anything
```



```
false
```

  is always false, the rest of the expression no longer matters.
- One of the advantages of short-circuit evaluation is that you can use && and || to prevent execution errors. If n were 0 in the earlier example, evaluating x %n would cause a "division by zero" error.

# Statements

- Statements in C++ fall into three basic types:
  - Simple statements
  - Compound statements
  - Control statements
- Simple statements are formed by adding a semicolon (;) to the end of a C++ expression.
- Compound statements (also called blocks) are sequences of statements enclosed in curly braces { }.
- Control statements fall into two categories:
  - Conditional statements that specify some kind of test
  - Iterative statements that specify repetition
- One can break a long line of code using a backslash (\). Most of the time compilers can recognize a broken line, but sometimes (e.g., in preprocessor directives) \ is needed.

# The if Statement

- The simplest of the control statements is the if statement, which occurs in two forms. You use the first form whenever you need to perform an operation only if a particular condition is true:

```
if (condition) {
    statements to be executed if the condition is true
}
```

*conditions are expressions that evaluate to a* **boolean** *value*

- You use the second form whenever you want to choose between two alternative paths, one for cases in which a condition is true and a second for cases in which that condition is false:

```
if (condition) {
    statements to be executed if the condition is true
} else {
    statements to be executed if the condition is false
}
```

# Common Forms of the if Statement

The examples in the book use only the following forms of the if statement:

Single line **`if`** statement

```
if (condition) statement
```

Multiline **`if`** statement with curly braces

```
if (condition) {
    statement
    . . . more statements . . .
}
```

**`if`**/**`else`** statement with curly braces

```
if (condition) {
    statements_true
} else {
    statements_false
}
```

Cascading **`if`** statement

```
if (condition_1) {
    statements_1
} else if (condition_2) {
    statements_2
. . . more else/if conditions . . .
} else {
    statements_else
}
```

# The ?: Operator

- In addition to the if statement, C++ provides a more compact way to express conditional execution that can be extremely useful in certain situations. This feature is called the ?: operator (pronounced question-mark-colon) and is part of the expression structure. The ?: operator has the following form:

*condition* **?** $expression_1$ **:** $expression_2$

Not allowed to be overloaded

- When C++ evaluates the ?: operator, it first determines the value of condition. If condition is true, C++ evaluates $expression_1$ and uses that as the value; if condition is false, C++ evaluates $expression_2$ instead.
- You can use the ?: operator to assign the larger of x and y to the variable max like this:

```
max = (x > y) ? x : y;
```

=

```
if (x > y) {
   max = x;
}
else {
   max = y;
}
```

# The switch Statement

The switch statement provides a convenient syntax for choosing among a set of possible paths:

```
switch ( expression ) {
   case v1:
      statements to be executed if expression = v1
      break;
   case v2:
      statements to be executed if expression = v2
      break;
   . . . more case clauses if needed . . .
   default:
      statements to be executed if no values match
      break;
}
```

When C++ executes a switch statement, it begins by evaluating expression, which must produce an integer-like value.

C++ then looks for a case clause that matches expression. If expression was equal to $v_2$, C++ would choose the second clause.

C++ evaluates the statements in a case clause until it reaches the break statement, which typically appear at the end of each clause.

If none of the values in the case clauses match the expression, C++ evaluates the statements in the default clause.

# Example of the switch Statement

```
int main() {
   int month;
   cout << "Enter numeric month (Jan=1): ";
   cin >> month;
   switch (month) {
     case 2:
       cout << "28 days (29 in leap years)" << endl;
       break;
     case 4: case 6: case 9: case 11:
       cout << "30 days" << endl;
       break;
     case 1: case 3: case 5: case 7: case 8: case 10: case 12:
       cout << "31 days" << endl;
       break;
     default:
       cout << "Illegal month number" << endl;
       break; // not necessary but a good habit
   }
   return 0;
}
```

If break is not included in one case, all statements following the case are also executed, until a break is reached or the end of the switch block.

# The while Statement

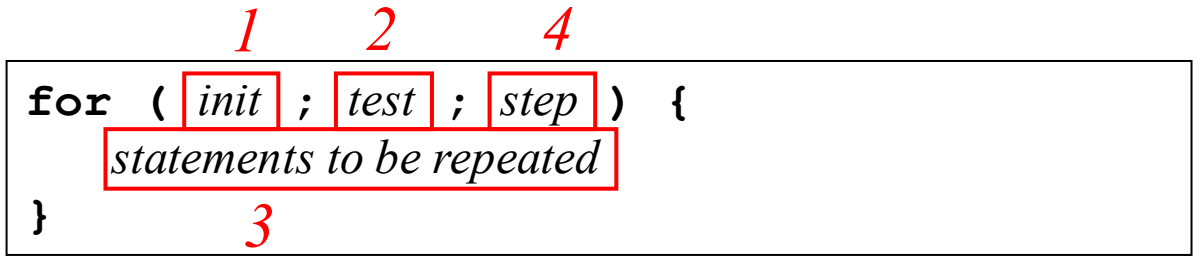
- The while statement is the simplest of C++'s iterative control statements and has the following form:

```
while ( condition ) {
      statements to be repeated
}
```

- When C++ encounters a while statement, it begins by evaluating the condition in parentheses.
- If the value of condition is true, C++ executes the statements in the body of the loop.
- At the end of each cycle, C++ reevaluates condition to see whether its value has changed. If condition evaluates to false, C++ exits from the loop and continues with the statement following the closing brace at the end of the while body.

# The for Statement

The for statement in C++ is a particularly powerful tool for specifying the control structure of a loop independently from the operations the loop body performs. The syntax looks like this:




C++ evaluates a for statement by executing the following steps:

1. Evaluate init, which typically declares a control variable.
2. Evaluate test and exit from the loop if the value is false.
3. Execute the statements in the body of the loop.
4. Evaluate step, which usually updates the control variable.
5. Return to step 2 to begin the next loop cycle.

# Comparing for and while

- The for statement

```
for ( init ; test ; step ) {
    statements to be repeated
}
```

is functionally equivalent to the following code using while:

```
init;
while (test) {
    statements to be repeated
    step;
}
```

- The advantage of the for statement is that everything you need to know to understand how many times the loop will run is explicitly included in the header line.

# Exercise: Reading for Statements

Describe the effect of each of the following for statements:

1. 
```
for (int i = 1; i <= 10; i++)
```
This statement executes the loop body 10 times, with the control variable i taking on each successive value between 1 and 10.

2. 
```
for (int i = 0; i < N; i++)
```
This statement executes the loop body N times, with i counting from 0 to N - 1. This version is the standard Repeat-N-Times idiom.

3. 
```
for (int n = 99; n >= 1; n -= 2)
```
This statement counts backward from 99 to 1 by 2's, for 50 times.

4. 
```
for (int x = 1; x <= 1024; x *= 2)
```
This statement executes the loop body with the variable x taking on successive powers of 2 from 1 up to 1024, for 11 times.

# C++ Programming Style Guide

- Style, also known as readability, is what we call the conventions that govern our C++ code.  The term Style is a bit of a misnomer, since these conventions cover far more than just source file formatting.
- Use common sense and be consistent.
- https://google.github.io/styleguide/cppguide.html
- Coding Style matters!




```
if(x==5) {
...
}
```

```
if(x=5) {
...
}
```

```
if(5==x) {
...
}
```

```
if(5=x) {
...
}//compiling error
```

# Unit Testing

- One of the most important responsibilities you have as a programmer is to test your code as thoroughly as you can. Although testing can never guarantee the absence of errors, adopting a deliberate testing methodology helps you to find at least some of the errors in your code.
- Whenever you write a module, it is good practice to create a test that checks the correctness of that module in isolation from the rest of the code. Such tests are called unit tests.
- You can use the assert macro from the <cassert> library to implement the unit-testing strategy. If the conditional expression in the assert macro is true, execution continues normally. If the expression is false, the assert macro prints a message identifying the source of the error and exits from the program.
- A macro is a fragment of code that has been given a name. Whenever the name is used, it is replaced by the contents of the macro.

# Example of the assert macro

```
int main()
{
    int x = 7;

    /*  Some big code in between and let's say x
       is accidentally changed to 9  */
    x = 9;

    // Programmer assumes x to be 7 in rest of the code
    assert(x==7);

    /* Rest of the code */

    return 0;
}
```

Assertion failed: x==7, file test.cpp, line 13. This application has requested the Runtime to terminate it in an unusual way. Please contact the application's support team for more information.